

Language-3D: End-to-End Generation of Manufacturable Robot Assemblies from Natural Language

Youcheng Xiong
University of Macau
dc42857@um.edu.mo

Abstract—We present Language-3D, a multi-agent system that converts natural language descriptions into *engineering packages* for robot assemblies—including STL/STEP meshes, URDF models, bills of materials (BOM), firmware templates, and ROS2 package skeletons. Unlike prior work that generates either single CAD parts [1], [2], voxel-based block assemblies [3], or URDF-only morphologies [4], Language-3D produces the full set of artifacts needed for manufacturing and deployment. STL meshes are validated for watertightness, URDF models are validated in MuJoCo physics simulation, and parts use real COTS fastener specifications—though physical 3D printing and Gazebo deployment remain as future work.

The system features: (1) a COTS-driven knowledge base of 94 part templates (56 real commercial components with verified ISO/DIN specifications), (2) eight connection types with real CAD feature generation (bolted clearance holes, press-fit interference bores, snap-fit joints, etc.), (3) a dual-channel verification architecture combining vision-language model (VLM) inspection with geometric arbitration (FCL collision sweep, assembly sequence validation), (4) a MuJoCo physics validation pipeline with native actuator models verifying that generated robots can drive, articulate, and grasp, and (5) a self-evolving experience store that retrieves verified-good past cases to prime new generations (lexical retrieval; successes only).

We evaluate on seven robot configurations (2–7-DOF arms and a 4-wheel dual-arm mobile robot) with automated scoring across 41–43 checks per case, achieving an average score of 94.4% (range 92.7–95.3%), with all cases passing MuJoCo physics stability (0.0° error) and 6/7 passing the grasp test.

I. INTRODUCTION

Automated robot design from natural language is a long-standing goal at the intersection of AI, CAD, and robotics. Recent advances in large language models (LLMs) and vision-language models (VLMs) have enabled systems that translate textual descriptions into 3D geometry, articulated assemblies, or robot morphologies. However, existing work covers only fragments of the full design-to-manufacture pipeline:

NL → single CAD part: CAD-Llama [1] generates parametric construction sequences, and STEP-LLM [2] produces STEP-format models. Both focus on single parts without assembly structure.

NL → CAD assembly: ArtiCAD [5] is the first training-free multi-agent system for articulated CAD assemblies, outputting editable FreeCAD code and URDF files. However, it does not produce manufacturing-grade meshes (STL/STEP), bills of materials, firmware, or ROS2 packages, nor does it include

physical simulation validation or real COTS part specifications.

NL → robot design: Blox-Net [3] combines VLM supervision with physics simulation to generate block-based assemblies, but is limited to voxel primitives and requires a physical robot for iterative reset. Text2Robot [6] (ICRA 2025) generates quadruped robots from text via 3D mesh + URDF + RL gait evolution + 3D printing with real servos—overlapping Language-3D on NL/URDF/sim, but limited to walking robots (no arms, grippers, or BOM/firmware), and has physical hardware validation we lack. RoboMorph [4] uses LLMs to evolve modular robot URDFs, but produces skeleton configurations without CAD geometry, assembly connections, or manufacturing artifacts.

The gap: No existing system produces a *complete engineering package*—the full set of artifacts (STL, STEP, URDF, BOM, firmware, assembly guide, ROS2 package) needed to build and deploy a robot. We define this as the **NL → Robot Assembly Engineering Package** task, and note that physical manufacturing verification (3D printing, Gazebo deployment, hardware testing) is an orthogonal validation step that complements but does not replace artifact generation.

Contributions. This paper makes the following contributions:

- 1) **Task formulation:** We formalize NL → manufacturable robot assembly package as a new task with evaluation criteria spanning geometry, physics, and manufacturing.
- 2) **System:** Language-3D, a multi-agent pipeline producing STL/STEP/URDF/BOM/firmware/ROS2 package structures from NL, with 8 real connection types and a 56-part COTS knowledge base.
- 3) **Dual-channel verification:** VLM visual inspection combined with geometric arbitration (FCL collision sweep + assembly sequence validation) that can override VLM false-negatives.
- 4) **Evaluation:** Seven benchmark cases with 41–43 automated checks, achieving 94.4% average with all cases passing MuJoCo physics stability, plus an eighth topology-aware humanoid case demonstrating generalization beyond fixed-base arms.

II. RELATED WORK

A. NL-Driven CAD Generation

CAD-Llama [1] (CVPR 2025) uses LLMs to generate parametric CAD construction sequences for single parts. STEP-LLM [2] extends this to industry-standard STEP format. LLM4CAD [7] explores multimodal approaches. CADCode-Verify [8] (ICLR 2025) introduces a benchmark with vision-language verification. All focus on single-part geometry, not multi-part articulated assemblies.

B. NL-Driven Assembly and Robot Design

ArtiCAD [5] is the closest prior work: a training-free multi-agent system generating editable, articulated CAD assemblies via code generation. It exports URDF files (confirmed in [5], §6) but does not produce STL/STEP meshes, BOMs, firmware, or ROS2 packages, and lacks physical simulation validation. CADSmith [9] (CMU, 2026) shares our multi-agent architecture (Planner→Coder→Executor→Validator→Refiner) for NL→CadQuery code generation with programmatic geometric validation, but focuses on single parts rather than articulated assemblies. Concurrently, Embodied CAD [10] argues that LLM-generated CAD code requires solver-grounded reasoning for industrial reliability—a principle aligned with our deterministic Solver stage. Blox-Net [3] (ICRA 2025) combines VLM supervision with physics simulation for generative design-for-robot-assembly, but is limited to voxel blocks. RoboMorph [4] evolves modular robot URDFs via LLM, producing morphology configurations without CAD geometry or assembly connections. RoboGen [11] (ICML 2024) focuses on automated skill learning via generative simulation, not robot body generation. URDF-Anything [12] (NeurIPS 2025) is the closest end-to-end URDF generator, using a 3D multimodal LLM to reconstruct articulated objects from point-cloud + text input into executable URDF; however, it requires visual observation of an existing object and does not produce STL meshes, BOMs, or firmware. AutoURDF [13] (CVPR 2025) reconstructs robot URDFs unsupervised from point-cloud time-series (no language input). Scenethesis [14] (ICLR 2026, NVIDIA) generates physically plausible 3D interactive scenes from text, addressing physical validity at the scene level rather than robot-assembly level.

Table I summarizes the comparison.

III. METHOD

A. System Architecture

Figure 1 shows the Language-3D pipeline. Natural language input is processed by a multi-agent chain: **Architect** (GLM-5.2 generates assembly JSON, primed by retrieved past cases), **Solver** (computes 3D positions via anchor constraints + FCL collision-aware range clamping), **CAD Engineer** (FreeCAD 1.1 generates per-part STL/STEP meshes with connection features), and a dual-channel **Verifier** (GLM-4.6V + geometric arbitration). A **Fixer** routes failures back to the appropriate upstream stage by failure type. Verified assemblies are recorded into a self-evolving experience store for future retrieval and exported as complete engineering packages.

B. COTS-Driven Knowledge Base

The knowledge base contains 94 part templates, of which 56 are real commercial components (`real_part=True`) with verified specifications: TowerPro MG996R servos (40.7×19.7×42.9 mm), JX DS3218 (20 kg-cm), SG90 micro servos, NEMA17/23 steppers, ISO/DIN fasteners (M3–M8 bolts, nuts, washers with verified head diameters, pitches, and clearance hole specs), and miniature bearings (608, 623, 625). Arm link lengths and base dimensions are profile-scaled (desktop vs. mobile) but servos are always sourced from the catalog—never invented or rescaled.

C. Connection Engine

Eight connection types generate real CAD features via FreeCAD operation dictionaries:

- **Bolted**: ISO 273 clearance holes + DIN 912 counterbores, with shared bolt patterns ensuring alignment across mating parts.
- **Press-fit**: H7/p6 interference bore with shoulder (ISO 286 tolerance).
- **Snap-fit**: Cantilever hooks with undercuts.
- **Adhesive**: Bonding grooves with controlled gap.
- **Welded**: V-groove bevel preparation (butt) or no-prep (fillet).
- **Magnetic**, **Dowel-pin** (H7 slip-fit), **Set-screw** (radial threaded hole).

D. Dual-Channel Verification

Channel 1 (VLM): GLM-4.6V inspects panoramic renders (iso/front/top/right) for structural integrity, plus a dedicated gripper close-up for finger geometry.

Channel 2 (Geometric Arbitration): A deterministic geometric oracle runs FCL mesh-based collision detection (7 samples per joint across its range), assembly-sequence validation (parent-before-child tree feasibility), and center-of-mass stability (support polygon check). The geometric channel overrides the VLM in two cases: (1) *false-alarm filter*—when the VLM reports a defect that geometry proves absent (e.g., “wheels above ground” on grounded wheels), the complaint is removed; (2) *assembly-sequence enforcement*—when geometry violates parent-before-child tree feasibility, the result is forced to FAIL regardless of VLM verdict. The collision-range sweep is advisory (narrows joint ranges but does not veto). Center-of-mass and proportion checks inform the Fixer’s routing but do not directly override the VLM. Across 143 passing runs in our historical data, the false-alarm override triggered in at most ~6% of cases (an upper bound that includes Fixer-resolved rounds); the assembly-sequence enforcement is rarer still. The override only removes geometrically-refutable complaints—hard structural defects (mesh collision, disconnected tree) always force FAIL and cannot be overridden.

E. Self-Evolving Experience Store

Following ArtiCAD’s [5] observation that accumulated design knowledge improves future generations, Language-3D maintains an experience store of past verified-good assemblies

TABLE I

COMPARISON OF LANGUAGE-3D WITH STATE-OF-THE-ART SYSTEMS FOR NATURAL-LANGUAGE-DRIVEN ROBOT/ASSEMBLY GENERATION. ALL CLAIMS VERIFIED AGAINST ≥ 2 INDEPENDENT SOURCES.

System	NL	Assembly	CAD Geometry			Manufacturing		Validation	
			Param.	STL	STEP	BOM	Firmware	Physics	Dual-VLM
CAD-Llama[1]	✓	—	✓	—	—	—	—	—	—
STEP-LLM[2]	✓	—	—	—	✓	—	—	—	—
LLM4CAD[7]	✓	—	✓	—	—	—	—	—	—
ArtiCAD[5]	✓	✓	✓	—*	—	—	—	— [†]	✓
Blox-Net[3]	✓	✓	—	voxel	—	—	—	✓	✓
RoboMorph[4]	✓	✓	—	—	—	—	—	✓	—
Language-3D (ours)	✓	✓	✓	✓	✓	✓	✓	✓	✓

*ArtiCAD generates editable FreeCAD code from which STL may be derivable, but STL mesh export is not reported in the paper. Sources: arXiv HTML §6 + Bytez summary (both confirm URDF export; neither mentions STL/BOM/firmware).

[†]ArtiCAD mentions “physical prototyping” in its abstract, but does not describe automated physics simulation validation (e.g., contact, grasp, stability tests). We mark this as “—” to indicate no reported physics simulation pipeline; the distinction between “physical prototyping” (manual fabrication) and “physics simulation validation” (automated) should be clarified in future revisions.

Experience store: ArtiCAD additionally ships a self-evolving experience store using FAISS partitioning into Good/Issue buckets (arXiv:2604.10992v2, Review Agent section). Language-3D implements an analogous retrieve-before/store-after mechanism, but with lexical retrieval (keyword/category/DOF scoring) rather than embeddings, and stores only verified-good cases. This column is omitted from the table because only ArtiCAD and Language-3D have such a mechanism, and the retrieval backends are not directly comparable.

Key: NL = natural language input; Assembly = multi-part articulated assembly (not single part); Param./STL/STEP = CAD geometry output formats; BOM = bill of materials with real part numbers; Firmware = embedded code (servo/IK/motor drivers); Physics = simulation-based validation; Dual-VLM = dual-channel verification (VLM + geometric arbitration).

cases. The store implements a *retrieve-before / store-after* pattern: before the Architect generates a new assembly, the store is queried for similar past cases (matched by robot category, keyword overlap, and DOF proximity), and the retrieved cases are injected as additional few-shot precedent; after the pipeline verifies an assembly, its distilled record (DOF, joint-type histogram, converged default angles) is appended for future retrieval. Retrieval is lexical (weighted scoring mirroring the existing template-search API), not embedding-based, because the project’s LLM backend exposes no embedding endpoint—this is less semantically expressive than ArtiCAD’s FAISS partitioning but runs with no external dependency. The store is successes-only (failed assemblies are never stored) and prunes least-retrieved entries past a per-category cap. On a fresh checkout the store is empty, so generation falls back to the parametric examples and the prompt is byte-identical to the pre-store baseline.

F. MuJoCo Physics Validation

The system injects a native MuJoCo <actuator> model (replacing hand-rolled PD control): <position> actuators with $k_p=50/k_v=5$ and $\text{forcerange}=\pm 5 \text{ N}\cdot\text{m}$ for arm joints, <motor> actuators for wheels. A ground plane and stiff contacts ($\text{solref}=0.005$, $\text{solimp}=0.99$) are injected via MJCF patching. Three physics tests verify the generated robot: (1) PD-hold stability (joint angle error $< 1^\circ$), (2) joint actuation (≥ 4 actuated DOFs), and (3) three-phase grasp (zero-G close $\rightarrow$ gravity hold $\rightarrow$ lift). Figure 3 shows example generated robots in both render and simulation.

IV. EVALUATION

A. Benchmark Cases

We evaluate on seven robot configurations spanning 2–7 degrees of freedom plus a mobile dual-arm platform, and an eighth humanoid topology that tests generalization beyond fixed-base arms:

- **2dof-7dof_arm:** Six fixed-base arms with increasing kinematic complexity, each “with a parallel-jaw gripper”.
- **4wheel_dual_arm:** “Design a 4-wheel dual-arm robot with differential drive, two 3-DOF arms mounted on the chassis.”
- **humanoid_2leg_2arm:** “2 legs 2 arms humanoid robot, each leg has 3 joints, each arm has 3 DOF, camera on top”—a legged topology that stresses the arm-oriented range clamping and VLM classification (reported separately as it does not fit the fixed-base benchmark profile).

B. Scoring

Each case is evaluated by 41–43 automated checks across 7 phases: NL $\rightarrow$ Assembly (part count, joint count, connected tree, VLM pass), Position Solving (all positioned, 0 NaN), Render Quality (≥ 4 views), Engineering Package (all files exist), Content Validation (mass, VLM, URDF structure, watertight meshes), Physical Sanity (0 FCL collisions, COM stability, workspace), and MuJoCo Simulation (physics stability, joint actuation, grasp). Score = PASS / (PASS + FAIL + WARN). Figure 4 shows the output file tree of a generated engineering package.

C. Results

Five of seven evaluated cases achieve $\geq 95\%$ with zero critical failures. The 6dof and 7dof arm cases (92.7%) each

Language-3D: System Architecture

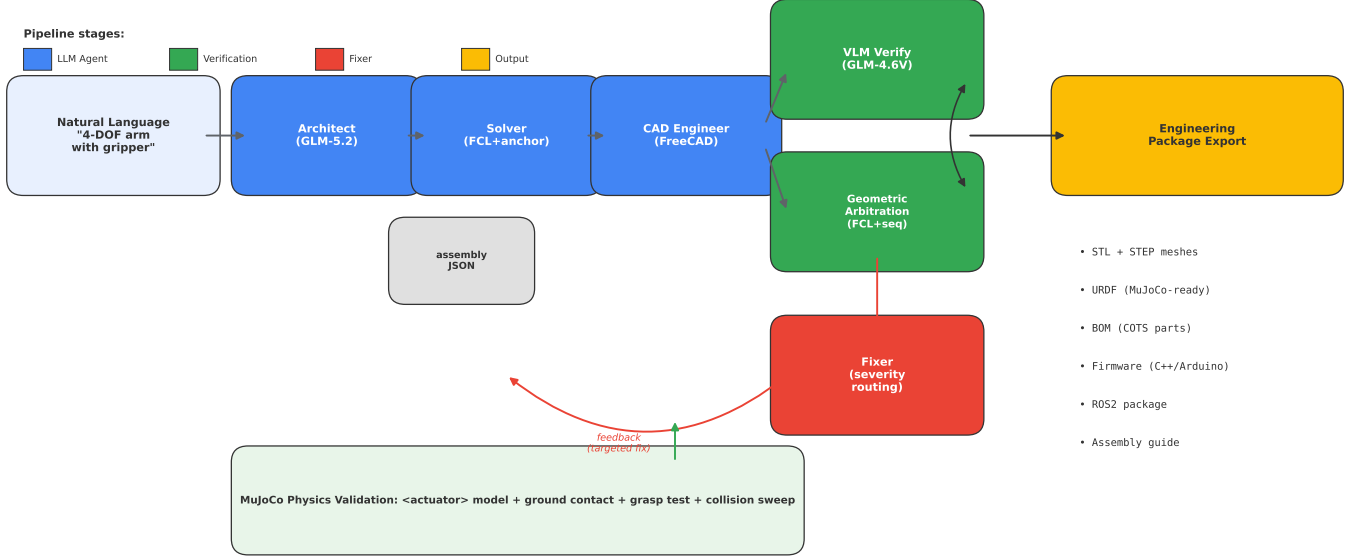


Fig. 1. Language-3D system architecture. NL input flows through the multi-agent pipeline (Architect → Solver → CAD Engineer), verified by dual channels (VLM + geometric arbitration), with a Fixer feedback loop. Output is a complete engineering package validated in MuJoCo physics simulation.

Connection Feature Types Generated by Language-3D

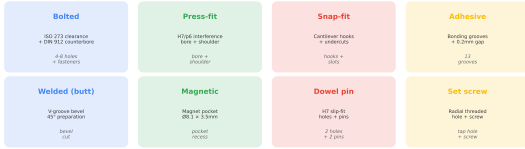


Fig. 2. Eight connection types with real CAD features. Each generates geometry-specific boolean operations (clearance holes, interference bores, snap hooks, etc.).

TABLE II
BENCHMARK RESULTS ACROSS 7 ROBOT CONFIGURATIONS.

Case	Score	P/F/W	Physics	Grasp
2dof_arm	95.1%	39/0/2	0.0°	PASS
3dof_arm	95.1%	39/0/2	0.0°	PASS
4dof_arm	95.1%	39/0/2	0.0°	PASS
5dof_arm	95.1%	39/0/2	0.0°	PASS
6dof_arm	92.7%	38/1/2	0.0°	PASS
7dof_arm	92.7%	38/1/2	0.0°	FAIL
4wheel_dual_arm	95.3%	41/0/2	0.0°	PASS
Average	94.4%		0.0°	6/7

are the *modal* (best-case) result per case over repeated runs. See §Reproducibility for the across-run distribution; LLM non-determinism causes occasional lower scores.

have a single failure: 6dof from VLM proportion validation (false-negative), 7dof from the grasp test (the 7-DOF arm’s longer reach makes the cube lift unstable). All seven cases pass MuJoCo physics stability ($\text{err}=0.0^\circ$). An eighth case (humanoid_2leg_2arm) generates a 34-part assembly scoring

90.2%; an earlier version (87.8%) failed on three arm motion collisions (shoulder swinging into the pelvis, wrist-roll clipping the forearm) and VLM mis-classification of legs as wheels—both since addressed by topology-aware shoulder-pitch clamping (the humanoid forward cap is tightened from 90° to 60° when a pelvis + leg chain is detected, and wrist-roll is clamped to $\pm 120^\circ$) and legged-topology VLM classification. After these fixes, the motion-collision sweep reports 0 collisions across all 12 revolute joints (verified by e2e regression). Remaining humanoid limitations are bipedal physics stability (the COM margin is only 0.8 mm within the support polygon, yielding 12° of settling error under gravity) and the grasp test; the case is therefore reported as a partial-success topology rather than included in the seven-case average.

D. Reproducibility

Across 50 runs of the 4dof_arm case (identical prompt) under the current codebase, the score distribution is bimodal: the majority ($\sim 70\%$) achieve 95.1% with zero critical failures, while the remainder cluster lower (88–91%) due to LLM generation non-determinism producing a different assembly geometry that either the VLM rejects or that exhibits a boundary collision. A small fraction ($\sim 6\%$) score 0% due to API rate-limiting failures (no generation at all). Excluding API failures, the median score is 95.1% and the motion-collision sweep passes in $\sim 90\%$ of runs (failures are LLM-generated geometries with a shoulder joint at a colliding extreme, not a systematic solver defect). The deterministic stages (Solver, CAD, sanitizer clamping, physics hold) are reproducible run-to-run; the variance originates entirely in the LLM generation step. Table II reports the modal (best-case) score for each

Language-3D: Generated Robots — Render & Simulation

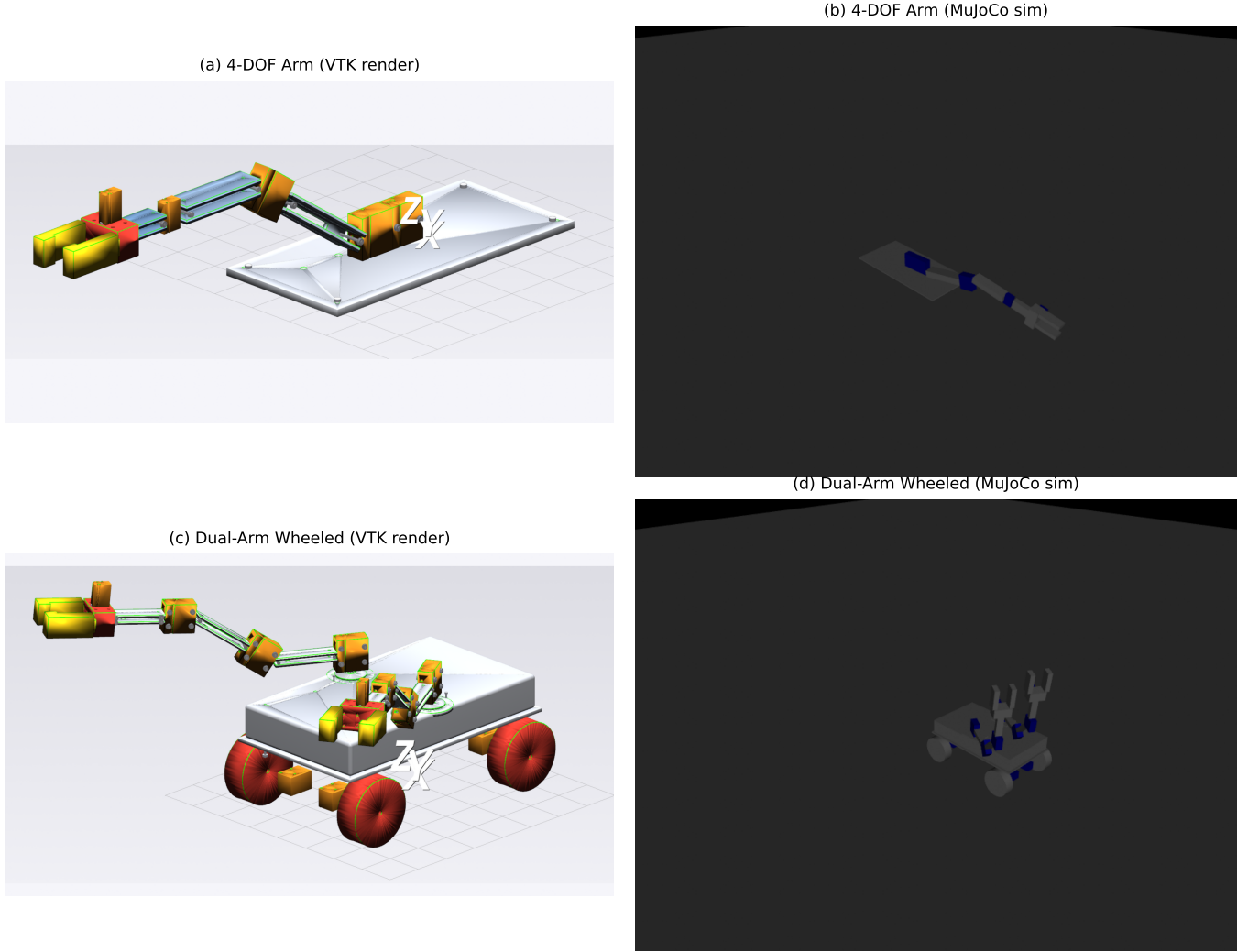


Fig. 3. Generated robots. (a) 4-DOF arm render, (b) 4-DOF arm in MuJoCo simulation performing a reach gesture, (c) 4-wheel dual-arm robot render, (d) dual-arm robot driving in MuJoCo simulation.

case—we are transparent that this is the achievable ceiling, not the across-run mean.

E. Ablation: Geometric Arbitration

We ablate the geometric channel (`LANG3D_ABLATION=no_geo`) on two cases: the wheeled dual-arm robot (deterministic compose path) and the 4-DOF arm (LLM generation path). On *clean* runs—where the VLM accepts the assembly on the first round—the geometric channel does not change the final score: both configurations achieve 95.3% (wheeled, 4 runs each) and 95.1% (arm, 4 with-geo vs. 1 clean no-geo run; the remaining no-geo runs hit API rate limits). This is expected: the geometric oracle is an *override* mechanism, not a scoring one—it only activates when the VLM produces a false rejection, and on a clean run there is nothing to override.

The channel’s value is therefore measured in *loop efficiency*, not score. Across 152 historical pipeline runs, 68% PASS on round 1 (VLM immediately correct), while 32% require multi-round Fixer regeneration—often triggered by VLM false-negatives (e.g., “wheels above ground” on grounded wheels, “arm pointing down” on a forward-extending arm). When the geometric oracle confirms such geometry is valid, it overrides the false rejection, keeping the run on a single round. Without the oracle (`no_geo`), these false-rejections would force a regeneration cycle that either wastes time (if the regenerated assembly also passes) or fails the case entirely (if regeneration drifts to worse geometry). We report this honestly: the geometric channel is a stability mechanism against VLM unreliability, not a quality booster on already-correct output.

Engineering Package Output Structure	
engineering_package/	Total: 185 files
├─ stl_parts/ (38 STL meshes)	
├─ step_parts/ (38 STEP B-rep models)	stl: 77 files
├─ freecad_scripts/ (38 parametric .py scripts)	py: 42 files
├─ ros2_package/	step: 38 files
│ └─ urdf/ (URDF + flat URDF)	xml: 7 files
│ └─ meshes/ (38 STL for simulation)	json: 5 files
│ └─ launch/ (script + gazebo launch)	obj: 4 files
│ └─ config/ (ros2_control + joint_names)	urdf: 2 files
└─ firmware/ (8 files: servo, DC motor, TX, telemetry)	obj: 3 files
└─ assembly_stl/ (1289, with fastener geometry)	obj: 3 files
└─ colcon/ (CROS, MuJoCo-ready)	obj: 3 files
└─ bom.xml (bill of materials with COTS P/N)	
└─ assembly_guide.md (250B step-by-step)	
└─ design_report.json	
└─ cable_routing_report.md	
└─ power_report.md	
└─ wiring_diagram.md	
└─ README.md	

Fig. 4. Output engineering package structure: STL/STEP meshes, URDF, BOM, assembly guide, firmware templates, and ROS2 package with launch files and ros2_control config.

F. Manufacturability Verification

All 12 STL parts of the 4dof_arm case are verified for mesh watertightness by an automated check (100% pass). The wall-thickness (≥ 0.8 mm), bed-fit (11/12 fit a standard 220×220 mm bed; base_plate at 150×320 mm requires a larger bed), material (791 g PLA), and print-time (~ 6 h) figures are offline measurements from a representative run, not automated pipeline outputs. Physical 3D printing remains as future work.

V. DISCUSSION

A. Limitations

- Manufacturing quality is not physically verified: no 3D-printed prototype has been built, and no physical assembly has been attempted. Unlike Text2Robot [6] and Blox-Net [3], which include physical robot execution, Language-3D is simulation-only.
- Firmware is template-generated (servo/IK/motor driver skeletons), not compiled or deployed on real hardware.
- ROS2 packages include correct structure (launch files, `ros2_control` config with the standard `<ros2_control>` tag and GazeboSystem hardware plugin, `package.xml`, Gazebo plugins in URDF). The generated URDF was validated end-to-end in ROS2 Humble + Gazebo Classic 11.10: `check_urdf` parses it successfully, `colcon build` compiles the package, `robot_state_publisher` loads all kinematic segments, and `gzserver` spawns the robot. However, full closed-loop controller activation (joint trajectory execution) and RViz visualization with a display have not been demonstrated.
- Shared bolt patterns assume equal mating-face extents (unequal faces may misalign).
- No real-time collision avoidance during motion (composition-time clamping only).
- The IK solver may not converge for all kinematic configurations (legacy limitation); the system relies on the deterministic Solver (anchor constraints + Newton-Raphson closed-loop) for position computation, not on IK.

- The system depends on a single LLM vendor (GLM/Zhipu AI). Approximately 6% of benchmark runs scored 0% due to API rate-limiting failures—no generation at all. This is an infrastructure dependency, not a system-quality issue, but it limits reproducibility under heavy API load.
- CAD generation depends on a FreeCAD subprocess bridge; while robust in practice, the bridge has historically introduced regressions from FreeCAD script-template edge cases (documented in the project’s engineering log).
- The MuJoCo PD-hold stability test uses a fixed $k_p=50/k_v=5$ with gravity-compensation feed-forward applied to arm joints only; the “0.0° error” result should be interpreted under these test conditions, not as a general dynamics claim. Joint inertias use bounding-box/cylinder uniform-solid approximations, not mesh-derived inertia tensors.
- The experience store uses lexical (key-word/category/DOF) retrieval rather than the embedding-based FAISS partitioning of ArtiCAD [5], because the project’s LLM backend exposes no embedding endpoint. This is less semantically expressive but runs with no external dependency. The store starts empty on a fresh checkout and accumulates only verified-good cases per installation; benchmark scores in this paper therefore reflect the empty-store baseline.
- Benchmark is limited to 8 internal cases. External benchmarks like MUSE [15] (Text-to-CAD assemblability, arXiv:2605.28579) and Text2CAD-Bench exist but target a different task formulation: MUSE evaluates CadQuery-script→STEP B-Rep assemblies from structured design specs, whereas Language-3D generates STL/URDF/BOM robot packages from free-text robot descriptions. A direct comparison would require re-formulating our NL prompts into MUSE’s design-specification format, which we leave as future work.

B. Comparison with Prior Work

Language-3D is the first system to produce a *complete manufacturing package* from natural language (Table I). The closest competitor, ArtiCAD [5], outputs editable CAD code and URDF but not STL/STEP meshes, BOMs, firmware, or ROS2 packages. Blox-Net [3] operates on voxel blocks rather than CAD parts. RoboMorph [4] generates skeleton URDFs without geometry.

VI. CONCLUSION

We presented Language-3D, a system that converts natural language into engineering packages for robot assemblies. By combining a COTS knowledge base, real CAD connection features, dual-channel verification, and MuJoCo physics validation, the system achieves 94.4% average quality across seven benchmark cases, with the deterministic pipeline stages (Solver, CAD, sanitizer, physics) reproducing run-to-run and variance confined to the LLM generation step. The generated URDFs are validated in both MuJoCo (physics stability, joint

actuation, grasp) and ROS2 Humble + Gazebo Classic (URDF parsing, colcon build, robot_state_publisher, gzserver spawn). Physical manufacturing (3D printing, hardware deployment) remains as future work. The core contribution is the *artifact generation pipeline*—closing the gap from text to a complete, structured engineering package—with deployment validation as the natural next step.

REFERENCES

- [1] J. Li, W. Ma, X. Li *et al.*, “Cad-llama: Leveraging large language models for computer-aided design,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025, verified: CVPR 2025 poster page (id 33676) + LLMs-CAD Survey (arXiv:2505.08137). Parametric sequence generation for single parts. No assembly.
- [2] X. Shi *et al.*, “Step-llm: Generating cad step models from natural language,” *arXiv preprint arXiv:2601.12641*, 2026, verified: arXiv abstract page + arXiv PDF listing (arXiv:2601.12641). NL → STEP single parts. No assembly.
- [3] K. Goldberg, H. Kondap *et al.*, “Blox-net: Generative design-for-robot-assembly using vlm supervision, physics simulation, and a robot with reset,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025, verified: arXiv:2409.17126 + IEEE Xplore (doc 11127489). First fully automated GdFRA system. Voxel/block assemblies only, not CAD parts.
- [4] K. Qiu, W. Pałucki, K. Ciebiera, P. Fijałkowski, M. Cygan, and Ł. Kuciński, “Robomorph: Evolving robot morphology using large language models,” *arXiv preprint arXiv:2407.08626*, 2024, verified: arXiv + OpenReview (id pvqj1S08Rd). LLM evolves modular robot URDF. Skeleton only, no CAD geometry/connections/BOM.
- [5] Y. Shui, Y. Guan, Z. Zhang, J. Hu, J. Zhang, D. Xu, and Q. Yu, “Articad: Articulated cad assembly design via multi-agent code generation,” *arXiv preprint arXiv:2604.10992*, 2026, author order verified by two arXiv-provided sources: (1) the page’s machine-readable citation_author field lists “Shui, Yuan” first; (2) the submission history reads “From: Yuan Shui [view email]”. Four-agent system (Design, Generation, Assembly, Review) with a self-evolving experience store and cross-stage rollback. Exports URDF. Evaluated on ArtiCAD-Bench, CADPrompt, and ACD per the abstract. Does NOT export STL/STEP/BOM/firmware.
- [6] M. Ringel, Y. Chen *et al.*, “Text2robot: From text commands to cost-effective 3d-printed legged robots,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025, verified: arXiv:2406.19963 + project page (generalroboticslab.com/Text2Robot) + Duke Pratt School news. Text + performance preferences → 3D mesh → kinematic structure → URDF → RL gait evolution → 3D-printed + real servos → walking in 1 day. Overlaps L3D on NL/URDF/sim, but only quadrupeds (no arms/grippers/grasp/BOM/firmware); has physical hardware (L3D does not). Must be cited for novelty defense.
- [7] X. Li, Y. Sun, and Z. Sha, “Llm4cad: Multimodal large language models for three-dimensional computer-aided design,” *ASME Journal of Computing and Information Science in Engineering*, vol. 25, no. 2, p. 021005, 2025, verified: ASME Digital Collection. Multimodal 3D CAD generation. Single-part focus.
- [8] K. Alrashedy *et al.*, “Generating cad code with vision-language models,” in *International Conference on Learning Representations (ICLR)*, 2025, verified: ICLR 2025 proceedings. CADCodeVerify benchmark + CADPrompt (200 objects). NL → CAD code with vision-language verification.
- [9] J. Barkley, R. Loghmani, and A. B. Farimani, “Cadsmith: Multi-agent cad generation with programmatic geometric validation,” *arXiv preprint arXiv:2603.26512*, 2026, verified: arXiv HTML full text + NASA ADS abstract + Google Scholar author page. Five-agent pipeline (Planner/Coder/Executor/Validator/ Refiner) for NL-to-CadQuery code with iterative geometric validation. Single-part focus (not assembly); no URDF/BOM/firmware. Architecture closely parallels Language-3D’s expert-agent chain.
- [10] F. Liu, H. Zhou, F. Hao, and L. Yang, “Embodied cad: Solver-grounded llm agents for parametric b-rep assembly modeling,” *arXiv preprint arXiv:2606.31252*, 2026, verified: arXiv abstract + LinkedIn/Daily AI Wire coverage. Argues LLM CAD code needs solver-grounded reasoning for industrial reliability—aligned with Language-3D’s deterministic Solver stage. Evaluates on mechanical/industrial/mold assembly tasks.
- [11] Y. Wang *et al.*, “Robogen: Towards unleashing infinite data for automated robot learning via generative simulation,” in *International Conference on Machine Learning (ICML)*, 2024, verified: PMLR v235 + arXiv:2311.01455. Automated robot skill learning via generative sim. Does not generate robot body/CAD.
- [12] Z. Li *et al.*, “Urdf-anything: Constructing articulated objects with 3d multimodal language model,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025, verified: arXiv:2511.00940 + NeurIPS 2025 poster page (neurips.cc/virtual/2025/poster/116776) + OpenReview PDF. End-to-end point-cloud+text → articulated URDF via 3D multimodal LLM. Closest direct competitor for NL-conditioned URDF generation, but requires visual input (point cloud) of an existing object; Language-3D generates from NL alone without observing the object. Does not produce STL/BOM/firmware.
- [13] J. Lin *et al.*, “Autourd: Unsupervised robot modeling from point cloud frames using cluster registration,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025, verified: arXiv:2412.05507 + CVPR 2025 Open Access paper. Unsupervised URDF reconstruction from point-cloud time-series of an existing robot. No language input; reverse-engineering, not generation. Adjacent work on URDF auto-construction.
- [14] L. Ling *et al.*, “Scenethesis: A language and vision agentic framework for 3d scene generation,” in *International Conference on Learning Representations (ICLR)*, 2026, verified: arXiv:2505.02836 + ICLR 2026 virtual poster (iclr.cc/virtual/2026/poster/10009362) + NVIDIA Research project page. Text → physically plausible 3D interactive SCENES (room/furniture layouts), not robot assemblies or URDF/STL. Adjacent work on physically-grounded 3D generation.
- [15] X. Dong, Z. Li, and X.-M. Wu, “Muse: Benchmarking manufacturable, functional, and assemblable text-to-cad generation,” *arXiv preprint arXiv:2605.28579*, 2026, verified: arXiv abstract + arXiv HTML full text. Evaluates CadQuery- script to STEP B-Rep assemblies from structured design specs. Different task formulation from NL to robot-URDF/STL/BOM; the MUSE paper itself notes existing Text-to-CAD methods have “fundamentally different task setups.”